AI-Augmented Catastrophe Modeling with An Intelligent Orchestration Workflow

Jainil Anilkumar Patel*, Tianyi Wang†, Shahid Hamid§, Mei-Ling Shyu*, Shu-Ching Chen‡
*School of Science and Engineering, University of Missouri–Kansas City, Kansas City, Missouri, USA
†International Hurricane Research Center, Extreme Events Institute, Florida International University, Miami, Florida, USA
§Department of Finance, College of Business, Florida International University, Miami, Florida, USA
‡Data Science and Analytics Innovation Center (dSAIC), University of Missouri–Kansas City, Kansas City, Missouri, USA
Email: japmyy@umkc.edu; tiwang@fiu.edu; hamids@fiu.edu; shyum@umkc.edu; s.chen@umkc.edu

***Abstract*—The Florida Public Hurricane Loss Model (FPHLM) estimates hurricane-induced insurance losses from exposure datasets submitted by multiple vendors. Although these datasets are expected to follow a canonical specification, in practice, they often contain heterogeneous schemas, malformed fields, and recurring data-quality defects that require extensive manual intervention. This paper presents an AI-augmented orchestration workflow for FPHLM that integrates agentic AI with explicit human oversight to improve preprocessing, monitoring, and verification while preserving operational safety and auditability. The proposed framework combines an LLM-guided preprocessing layer, a retrieval-augmented generation (RAG)-based auto-verifier, and an anomaly-detection module for identifying unusual policy-file behavior and pipeline outcomes. Rather than treating large language models as a mere coding aid, the system uses them as operational components within a constrained multi-agent protocol to support structured data cleaning and grounded verification. The contribution of this paper is the enhancement of automating the cleaning and verification workflow in the CatOP framework, a dynamic alerting module, and outlier detection using the DBSCAN for detecting abnormal processing patterns that may indicate data or workflow inconsistencies.**

***Index Terms*—Agentic AI, catastrophe modeling, Florida Public Hurricane Loss Model**

I. Introduction

The Florida Public Hurricane Loss Model (FPHLM) [1] is a catastrophe-modeling framework designed to estimate hurricane-induced insurance losses from exposure datasets submitted by insurers. The exposure files, provided in CSV format, contain building-level attributes such as insured values, premiums, deductibles, construction characteristics, and location information that are used as key inputs for downstream loss estimation. Given these inputs, FPHLM estimates potential financial loss [2], [3] under plausible hurricane scenarios by combining hazard simulation with Monte Carlo-based loss computation. In this process, wind fields are pre-generated from historical hurricane behavior, and losses are estimated at the building level.

Despite the existence of a canonical input specification, vendor-submitted exposure files often deviate from the required format in ways that complicate reliable model execution. In practice, these deviations include heterogeneous schemas, malformed or missing values, delimiter-related parsing failures, inconsistent identifiers, and other recurring data-quality defects. Such issues can interrupt dataset creation, propagate errors into downstream components, and substantially increase the amount of manual effort required from processors. Historically, these defects were handled through expert review and correction, producing logs and sample files that captured recurring failure patterns and prior remediation actions. These historical records later provided the foundation for the deterministic cleaning module used in CatOP [4], both by informing reusable cleaning rules and by supporting the construction of test cases for validating the resulting logic.

This paper presents an AI-augmented orchestration workflow for FPHLM that reuses historical operational knowledge to improve three critical stages of the pipeline: exposure-file preprocessing, workflow monitoring, and output verification. The proposed approach integrates agentic artificial intelligence (AI) with explicit human oversight, using LLM components as operational elements within a constrained multi-agent framework to support structured cleaning assistance, grounded reasoning over historical cases, and controlled exception handling. A retrieval-augmented generation (RAG)-based auto-verifier compares current outputs against previously reviewed evidence, while anomaly-detection mechanisms identify unusual file characteristics and abnormal processing behavior. Because exposure files contain sensitive insurance information, the framework operates under strict guardrails, including local model hosting, job isolation, absence of internet connectivity, and restricted execution boundaries. These safeguards enable the use of LLM-based reasoning in a setting where data confidentiality, operational trustworthiness, and reproducibility are critical, while ensuring that human review remains the essential control layer throughout the workflow.

CatOP (Catastrophe Model Operation Platform) is a Java-based web portal that manages user interactions and updates the system database. A separate Java based TaskRunner service continuously monitors selected database tables and triggers backend processing steps. In addition, a Python-based watcher runs as a third orchestration component for asynchronous operational monitoring and event handling. TaskRunner dispatches core processing stages such as dataset creation, WSC (Wind Speed Correction), and ILM (Insuarance Loss Model) execution, while watcher components handle notifications and coordinate bounded AI assistance when failures or diagnostically meaningful exceptions are detected. This separation keeps core

computation in the task runner and places monitoring and agent orchestration in a dedicated watcher layer.

The rest of this paper is organized as follows. Section II reviews related work, and Section III presents the agentic AI design for FPHLM. The proposed methodology is introduced in Section IV. Finally, Section V summarizes this paper.

II. Related Work

Research on data cleaning has long recognized that dirty data arises from heterogeneous schemas, missing values, and integration errors, and that cleaning must be treated as part of the broader data-management process rather than as an isolated preprocessing step [5]. Subsequent work broadened this perspective from hand-engineered rules to composite error-detection and repair strategies. In [6], the authors showed that no single detector dominates in all error types and argued for holistic combinations of tools. ActiveClean [7] demonstrated that cleaning can be integrated iteratively with downstream statistical modeling, while HoloClean [8] casts repairs as probabilistic inference over multiple signals. Our work builds on this literature, but focuses on regulated exposure-data processing, where cleaning decisions must remain auditable and compatible with deterministic downstream catastrophe-model execution.

A related line of work concerns monitoring long-running data workflows and detecting workload-conditioned delays. Quantile regression provides a principled way to estimate upper conditional envelopes rather than solely conditional means [9]. This is particularly useful in operational environments where fixed time thresholds are misleading because expected duration varies with workload size. Our runtime monitoring for dataset creation and WSC tasks adopts this perspective by deriving workload-adjusted alert thresholds from empirical scaling behavior. In parallel, log-based anomaly detection has moved from rule-based inspection to learned sequence models. DeepLog [10] modeled normal log sequences with recurrent neural networks, and LogBERT [11] extended this direction using transformer-based self-supervision. In contrast, our system combines structured preprocessing summaries, manipulation logs, and workload-aware envelopes to support operator-facing diagnosis and manual escalation.

At the application level, prior FPHLM research has addressed system integration and software infrastructure for public hurricane loss modeling. Chen et al. [1] described the challenges of multidisciplinary system integration for the Florida Public Hurricane Loss Model (FPHLM), while the software-system perspective for the insurance loss projection was presented in [4]. Our work continues this operational line but shifts the emphasis toward data cleaning, log interpretation, anomaly detection, and automated monitoring across dataset creation, WSC, ILM, and verification [2], [3], [12].

Finally, our human-supervised design is informed by prior work on human-AI interaction and tool-augmented language models. Amershi et al. [13] distilled widely applicable principles for human-AI interaction, emphasizing clear capabilities, user control, and careful communication of system behavior. ReAct [14] showed how language models can interleave reasoning and tool use, while retrieval-augmented generation (RAG) [15] demonstrated how external evidence can improve grounding and provenance in knowledge-intensive tasks. LLaMA [16] further showed the practicality of open foundation models in controlled deployment settings. Our contribution differs from general-purpose agentic systems by constraining model access to bounded artifacts, deterministic scripts, explicit evidence, and human approval gates before high-consequence actions are taken.

III. Agentic AI in FPHLM

In this work, an *agent* denotes a LangChain-based orchestration component grounded in a locally hosted LLaMA 70B foundation model and deployed within a strictly sandboxed execution environment. Unlike general-purpose autonomous agents, the proposed agent is intentionally constrained to satisfy the privacy, reliability, and governance requirements of catastrophe-modeling workflows that operate on sensitive insurance data. Its execution context is fully job-isolated: the agent has no Internet access, no cross-job memory, and no visibility into files, outputs, or state beyond the currently assigned task. File-system permissions are restricted to a predefined working directory, and both the readable inputs and writable outputs are explicitly bounded in advance.

A central design principle of the framework is *least-privilege data exposure*. Rather than granting the model unrestricted access to full raw policy files, the system first filters and structures the available evidence so that the agent receives only the minimum task-relevant context required for reasoning. Depending on the application stage, this context may consist of schema summaries, flagged rows or columns, cleaner logs, header-review results, structured verification sections, or synthetic representations used during logic development. In this way, the model is guided by information about how the data are structured, what constitutes normal behavior, and where anomalies have been detected, without unnecessarily exposing the full underlying dataset. This bounded-context strategy reduces privacy risk while also improving the relevance and traceability of agent decisions.

This constrained design constitutes a key novelty of the framework. Rather than granting unrestricted autonomy, the agent is permitted to operate only within narrowly defined guardrails that support auditable and reproducible decision assistance. It cannot install external dependencies, traverse unrelated directories, or modify protected artifacts through deletion, renaming, or relocation. When code generation is enabled, the agent may propose candidate logic or provide reasoning support, but execution remains subject to explicit human approval. Similarly, when database access is available, the agent is limited to writing only to predefined agent-output fields, such as structured evidence records, decision indicators, or references to generated artifacts, while deterministic pipeline fields remain immutable.

These guardrails transform the agent from a general conversational model into a trustworthy operational component for

FPHLM. The resulting architecture enables human-in-the-loop AI assistance for data cleaning, verification, and exception analysis while preserving strong guarantees on data privacy, job isolation, and controlled execution. In this sense, the contribution is not merely the use of a large language model, but the design of a secure agentic protocol that makes foundation-model reasoning usable in a high-stakes catastrophe-modeling environment.

IV. Methodology

Our methodology is organized as a sequential operational workflow for FPHLM. It begins with preprocessing, where historical defect patterns are encoded into a deterministic cleaner and residual ambiguities are handled through a bounded Cleaner Agent. It then proceeds to automated pipeline orchestration, in which historically recurring execution paths are converted into default chained processing steps monitored by watcher components. The methodology next incorporates historical runtime modeling for workload-aware notifications and a dataset-level anomaly-detection module for identifying structurally atypical inputs. Finally, after ILM execution, a RAG-based auto-verifier analyzes verification logs using retrieved human-reviewed precedents and explicit failure checks. This staged design enables preprocessing, execution monitoring, anomaly detection, and verification to operate within a unified, privacy-preserving, and human-in-the-loop framework.

A. Reuse of Historical Operational Knowledge

Exposure files submitted by multiple vendors frequently contain recurring defects such as schema mismatches, malformed values, delimiter-related row corruption, missing identifiers, and address-format anomalies. To make these failures reusable rather than case-specific, processors recorded observed issues and corrective actions in structured operational notes, which were then used to develop a deterministic cleaning module that encodes recurring remediation patterns as auditable rules for policy-file normalization and repair.

Large language models (LLMs) were used only as development assistants to translate well-documented historical cleaning actions into maintainable Python code, while production execution remained fully deterministic. To support privacy-preserving development and testing, we constructed synthetic policy-file examples that reproduce real-world defect patterns without exposing sensitive raw data and used them to build a `pytest`-based regression suite. Together, these historical records, synthetic examples, and automated tests provide a reproducible and privacy-preserving foundation for deterministic cleaners aligned with production failures.

1) LLM-guided remediation when deterministic cleaning fails: Deterministic preprocessing resolves most recurring policy-file defects, but some submissions still contain residual structural ambiguities that cannot be repaired safely with fixed rules alone. For example, if a malformed row contains an extra spurious cell, merging the excess content into the address column may still fail to restore schema alignment, so that subsequent fields are shifted and values appear under incompatible column types. For these cases, we introduce a *Cleaner Agent* as a human-supervised remediation layer applied after the deterministic cleaning stage. Operating on the cleaned file and the structured findings produced upstream, the agent examines unresolved issues such as extra columns, missing or shifted headers, suspicious header matches, schema-width inconsistencies, and PR (Personal Residential)/CR (Commercial Residential) mode mismatches. It is particularly useful when embedded commas or similar formatting defects cause a row to be parsed into an incorrect number of fields, leading to downstream load failures.

Given the bounded evidence assembled by the pipeline, the agent infers the most likely schema interpretation and proposes a remediation plan for operator review. Typical recommendations include dropping unsupported columns, reconciling noncanonical headers, repairing row misalignment, or applying narrowly targeted value corrections. When appropriate, the agent also generates candidate Pandas code to implement the proposed repair. This code is advisory rather than autonomous and serves as a controlled mechanism for handling rare failure modes that are difficult to capture through static rules.

The remediation workflow proceeds in explicit stages. The agent first profiles the cleaned file and summarizes unresolved anomalies. After operator approval, it generates candidate repair code within a fixed execution scaffold. If that code is approved, it is executed only on a copy of the cleaned file, producing a separate artifact, `clean-by-agent-file`, while leaving the original cleaned file unchanged. The system then presents execution results and file differences for inspection, and the reviewed artifact is promoted only through an additional operator action. This makes the agent a supervised exception-handling component rather than an autonomous modifier of production data. To support rapid experimentation, we implemented this remediation layer as a local Streamlit application backed by a reusable Python runtime. This design complements deterministic preprocessing: rule-based cleaning handles common defects, while the Cleaner Agent provides auditable assistance for rare or ambiguous structural failures requiring contextual interpretation.

The Cleaner Agent changes the role of the processor from manual cell-level editing to supervised review and approval. In the earlier workflow, when a defect occurred in a specific row, the processor often had to open the CSV in a spreadsheet tool, navigate to the affected location, inspect malformed cells, manually delete or correct the offending values, and then rerun the cleaning step. In the proposed workflow as shown in Figure 1, the agent instead analyzes the post-cleaning findings, proposes a bounded repair plan, generates candidate code, and produces a separate repaired artifact for comparison. The human remains in control, but the required effort shifts from low-level file manipulation to higher-level validation of the proposed remediation. This reduces cognitive load, improves consistency, and provides a more auditable record of what was changed, including the proposed action, generated code, execution result, and approval trail.

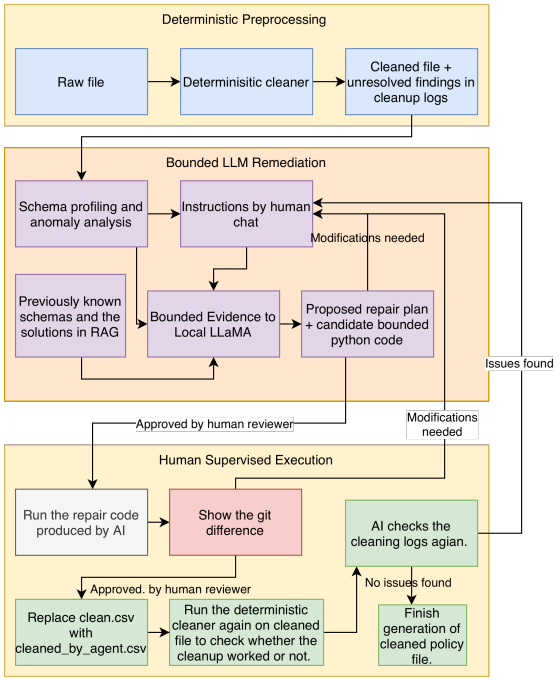

Fig. 1. Cleaner Agent flow diagram.

2) Automated Pipeline Orchestration: Historical CatOP usage showed that many ILM-related requests followed a repeated and largely standardized execution pattern. By reviewing these historical requests and the way they were repeatedly handled in practice, we identified a common execution template that could be automated safely. As shown in Figure 2, this made it possible to encode a historically derived default pipeline in which, once prerequisite conditions are satisfied, the next stage is triggered automatically without requiring processors to click through each intermediate action.

The resulting orchestration model includes two classes of automation. First, completion-triggered functions execute immediately after a preceding stage terminates successfully and initiate the next defined action in the workflow. Second, watcher-based components continuously monitor for newly created datasets, newly generated intermediate artifacts, or newly satisfied stage-completion conditions, and then dispatch the next eligible processing step. This hybrid design supports both immediate chained execution and continuous background detection of newly ready work.

Importantly, automation does not replace the older manual flow in all cases. Some company requests require nonstandard outputs, special handling, or deviations from the common processing template. For such cases, the original operator-driven workflow remains available. Thus, the automated pipeline should be understood as a historically derived default path for standard jobs, while the legacy manual path is preserved for exceptional or customized processing needs.

Rather than persisting step-completion markers directly in the dataset table, CatOP maintains runtime orchestration state in memory to track which stages have completed for datasets under active processing. This separates workflow-control concerns from persistent dataset content and reduces schema-level coupling between operational logic and stored business records.

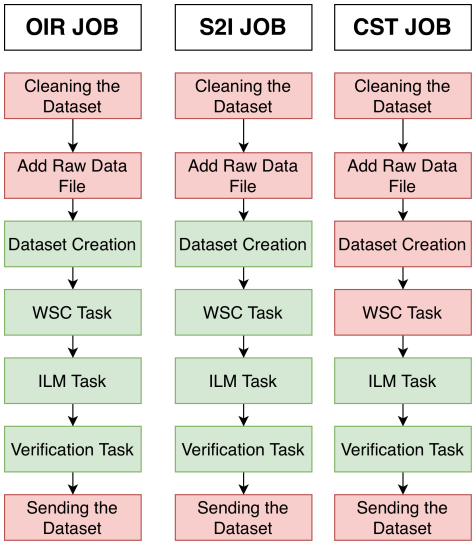

Fig. 2. Historically derived default CatOP processing pipeline for standard OIR (Office of Insurance Regulation), S2I (Service to Industry), and CST (Catastrophe Stress Test) recurring jobs. Green boxes denote automatically executed stages, while red boxes denote stages that still require human action or manual intervention.

B. Historical Runtime Modeling for Notifications

Because CatOP executes multiple stages automatically once prerequisite conditions are satisfied, intermediate failures may not be immediately visible to the processor. To address this issue, we developed a watcher-based notification module that monitors dataset creation, WSC, and ILM execution and alerts the assigned processor when runtime deviates substantially from historically expected behavior.

The notification strategy is grounded in historical runtime data from previously completed jobs. For each stage, policy count is used as the primary workload variable, and historical duration is modeled as a function of workload. The resulting workload-adjusted upper reference boundary is then used to distinguish genuinely abnormal delay from normal long-running behavior caused by large portfolios.

Figure 3 shows the historical runtime envelope used for dataset-creation notifications. This stage depends on supporting services such as geocoding and preprocessing utilities, so infrastructure-side degradation can delay completion even when the input itself is valid.

For WSC, the watcher uses the following workload-adjusted reference lines:

$$\log(h) = -2.3623 + 0.2277\log(p), \tag{1}$$

$$\log(h_{0.95}) = -1.3618 + 0.2277\log(p), \tag{2}$$

where p denotes policy count and h denotes runtime in hours. A WSC task is flagged as abnormally slow when its observed runtime exceeds the upper reference line.

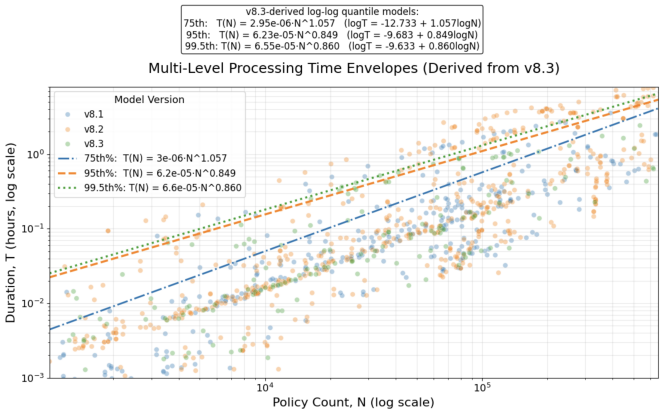

Fig. 3. Historical dataset-creation runtime as a function of policy count. The workload-adjusted upper boundaries are used by the watcher to distinguish normal long-running jobs from abnormal delay.

For ILM, the corresponding historical runtime model is:

$$\log(h) = -7.5033 + 0.8524\log(p), \tag{3}$$

$$\log(h_{0.95}) = -6.8759 + 0.8524\log(p), \tag{4}$$

where p denotes policy count and h denotes runtime in hours. An ILM job is escalated when its runtime exceeds the workload-adjusted upper boundary.

Together, these historical runtime models provide a unified notification mechanism for automated FPHLM processing. By grounding alerts in prior workload-dependent behavior rather than fixed thresholds, the module reduces unnecessary notifications while still surfacing jobs that are likely stalled, degraded, or in need of manual inspection. These model fitting was done using all the available data, and we expects the false positive rate of 5 percent in the production.

1) Dataset-level anomaly detection: We identified anomalous policy files by constructing dataset-level feature vectors from parsed input summaries and preprocessing logs, where each dataset was represented by summary statistics describing its overall structural profile rather than by individual policy records. Features were derived from parsed summaries and preprocessing logs, with DBSCAN (Density-Based Spatial Clustering of Applications with Noise) parameters chosen empirically. These features capture properties such as attribute distributions, categorical composition, and other file-level characteristics that reflect whether a dataset is consistent with the historical population of successfully processed files. As shown in Figure 4, the resulting feature vectors were projected into a lower-dimensional space using Principal Component Analysis (PCA), after which DBSCAN was applied to distinguish the dominant dense region of historically normal datasets from isolated outliers flagged for review. DBSCAN was selected because it can recover dense regions of historically normal behavior while explicitly labeling isolated observations as noise, without requiring the number of clusters to be specified in advance. In this setting, datasets assigned to the dominant dense region were treated as historically consistent, whereas isolated datasets were flagged for review as potential anomalies. In addition to these distribution-based signals, unusually large deletion behavior observed in preprocessing logs was used as a complementary indicator, since excessive record removal often reflects deeper data-quality problems.

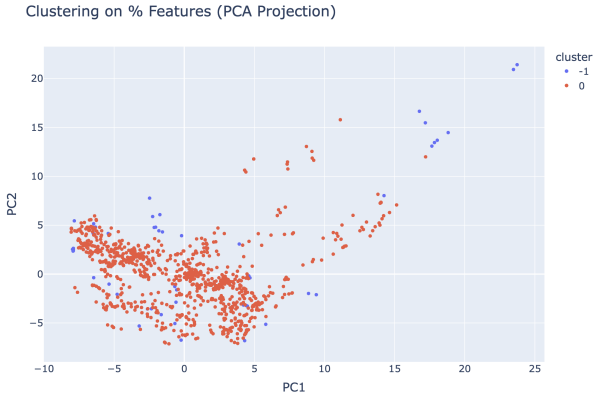

Fig. 4. PCA projection of dataset-level policy-file features derived from parsed input summaries and preprocessing logs. DBSCAN identifies the dominant dense region as the historical norm and flags isolated datasets as outliers for review.

To support interpretability, the anomaly detector does not stop at assigning an outlier label. For each flagged dataset, it compares the file against nearby historical inliers and ranks the summary attributes that differ most from the normal population. These explanations can highlight unusual construction-type proportions, abnormal county-level distributions, over-representation of rare categories, and other summary-level inconsistencies, helping processors focus on the aspects of the dataset most likely to require diagnosis or correction.

C. Verification and RAG-Based Auto-Verifier

After ILM execution, a monitoring component checks whether the verification task is progressing normally and alerts the processor when abnormal delay suggests an infrastructure-side failure. Once ILM finishes, the resulting verification logs are collected as evidence for automated review.

A central challenge in verification is that some degree of mismatch is operationally normal. To support this decision, we developed a retrieval-augmented auto-verifier that reuses previously reviewed verification logs as historical reference knowledge. During ingestion, human-verified logs are parsed into structured sections and stored as retrieval items together with tolerance summaries that capture previously accepted deviations, such as policy-count differences, analysis-count differences, and reviewable warning patterns. At verification time, the current log is matched against similar historical sections, and the corresponding tolerance summaries are used to determine whether observed mismatches remain within historically accepted bounds. In parallel, explicit rule-based checks detect strong failure indicators such as missing files, error markers, tracebacks, and fatal execution signals.

Unlike a purely generative reviewer, the verifier does not assess each log in isolation or treat all mismatches as equally

significant. Its decision process is grounded in retrieved human-verified precedents, so that small deviations previously accepted in practice can be distinguished from genuinely abnormal outcomes. This is particularly important in operational settings where minor count differences or warning patterns may be routine, whereas missing artifacts, fatal errors, or structurally inconsistent sections indicate true verification failure. By combining retrieval-grounded tolerance assessment with explicit rule-based failure detection, the verifier provides a more stable and auditable decision process than either heuristic thresholding or unconstrained LLM judgment alone, as illustrated in Figure 5.

To support downstream integration, the verifier produces machine-readable JSON, rather than free-form text. The section-level assessment includes fields such as is_good, summary, confidence, needs_human_review, issues, and tolerated_differences, which are then aggregated into a final report indicating whether the log is acceptable and which sections require review.

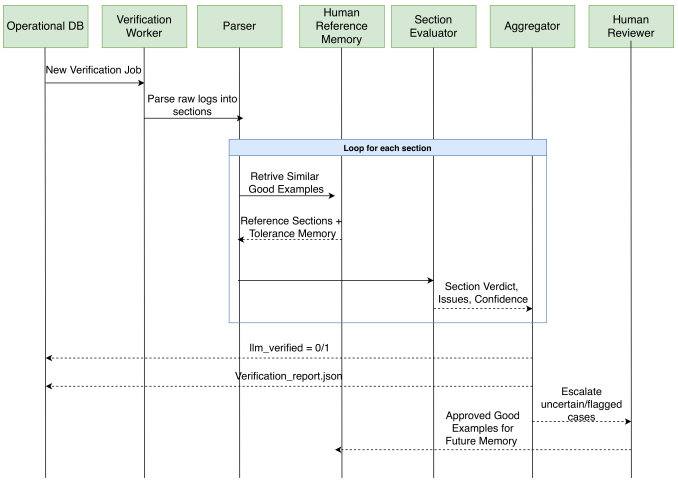

Fig. 5. CatOP *Verify Jobs* flow diagram to review completed automation tasks after ILM execution.

V. Conclusion

This paper presented an AI-augmented orchestration workflow for FPHLM that improves preprocessing, monitoring, and verification while preserving human oversight and data privacy. By combining deterministic cleaning, bounded LLM-guided remediation, historically derived automation, watcher-based pipelining, and a local RAG-based verifier, the framework reduces manual effort and shortens end-to-end processing time. In particular, processors no longer need to wait for one stage to finish before manually triggering the next, so the time required for dataset cleaning, pipeline execution, and verification is reduced while operational consistency and auditability are improved. Likewise, because the Cleaner Agent shifts remediation from manual cell-level editing to supervised review, a single operator can oversee multiple concurrent modeling jobs rather than being limited to the strictly sequential, file-by-file workflow required in the earlier process. More broadly, the contribution of this work lies not only in introducing LLM-based assistance into FPHLM, but in showing how such assistance can be deployed in a bounded, auditable, and human-supervised manner for high-stakes catastrophe-modeling operations.

VI. Acknowledgment

This work is partially supported by the Florida Office of Insurance Regulation (OIR) under "Hurricane Loss Projection Model" project. While the project is funded by Florida OIR, OIR is not responsible for the content of this paper.

References

[1] S.-C. Chen, M. Chen, N. Zhao, S. Hamid, K. Chatterjee, and M. Armella, "Florida public hurricane loss model: Research in multi-disciplinary system integration assisting government policy making," *Government Information Quarterly*, vol. 26, no. 2, pp. 285–294, 2009.

[2] S. S. Hamid, J.-P. Pinelli, S.-C. Chen, and K. Gurley, "Catastrophe model based assessment of hurricane risk and estimates of potential insured losses for the state of florida," *ASCE Natural Hazards Review*, vol. 12, no. 4, pp. 171–176, 2011.

[3] S. Hamid, B. M. G. Kibria, S. Gulati, M. Powell, B. Annane, S. Cocke, J.-P. Pinelli, K. Gurley, and S.-C. Chen, "Predicting losses of residential structures in the state of florida by the public hurricane loss evaluation model," *Statistical Methodology*, vol. 7, no. 5, pp. 552–573, 2010.

[4] Y. Tao, T. Wang, A. Sun, S. S. Hamid, S.-C. Chen, and M.-L. Shyu, "Florida public hurricane loss model: Software system for insurance loss projection," *Software: Practice and Experience*, vol. 52, no. 7, pp. 1736–1755, 2022.

[5] E. Rahm and H. H. Do, "Data cleaning: Problems and current approaches," *Bulletin of the IEEE Computer Society Technical Committee on Data Engineering*, vol. 23, no. 4, pp. 3–13, 2000.

[6] Z. Abedjan, X. Chu, D. Deng, R. C. Fernandez, I. F. Ilyas, M. Ouzzani, P. Papotti, M. Stonebraker, and N. Tang, "Detecting data errors: Where are we and what needs to be done?" *Proceedings of the VLDB Endowment*, vol. 9, no. 12, pp. 993–1004, 2016.

[7] S. Krishnan, J. Wang, E. Wu, M. J. Franklin, and K. Goldberg, "Activeclean: Interactive data cleaning for statistical modeling," *Proceedings of the VLDB Endowment*, vol. 9, no. 12, pp. 948–959, 2016.

[8] T. Rekatsinas, X. Chu, I. F. Ilyas, and C. Ré, "Holoclean: Holistic data repairs with probabilistic inference," *Proceedings of the VLDB Endowment*, vol. 10, no. 11, pp. 1190–1201, 2017.

[9] R. Koenker and J. Gilbert Bassett, "Regression quantiles," *Econometrica*, vol. 46, no. 1, pp. 33–50, 1978.

[10] M. Du, F. Li, G. Zheng, and V. Srikumar, "Deeplog: Anomaly detection and diagnosis from system logs through deep learning," in *ACM SIGSAC Conference on Computer and Communications Security*, 2017, pp. 1285–1298.

[11] H. Guo, S. Yuan, and X. Wu, "Logbert: Log anomaly detection via bert," *arXiv preprint arXiv:2103.04475*, 2021.

[12] M.-L. Shyu, S.-C. Chen, K. Sarinnapakorn, and L. Chang, *Principal Component-based Anomaly Detection Scheme*. Foundations and Novel Approaches in Data Mining, Springer-Verlag, 2006, pp. 311–329.

[13] S. Amershi, D. Weld, M. Vorvoreanu, A. Fourney, B. Nushi, P. Collisson, J. Suh, S. Iqbal, P. N. Bennett, K. Inkpen, J. Teevan, R. Kikin-Gil, and E. Horvitz, "Guidelines for human-ai interaction," in *CHI Conference on Human Factors in Computing Systems*, 2019, pp. 1–13.

[14] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "React: Synergizing reasoning and acting in language models," in *International Conference on Learning Representations*, 2023.

[15] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. tau Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive nlp tasks," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459–9474.

[16] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, A. Rodriguez, A. Joulin, E. Grave, and G. Lample, "Llama: Open and efficient foundation language models," *arXiv preprint arXiv:2302.13971*, 2023.